

Shamoni

Build an immersive, highly interactive, scroll-driven landing page using React, Vite, Tailwind CSS (v4), and `motion/react` (Framer Motion).

Please set up the application with the exact files, dependencies, URLs, CSS variables, and mathematical Framer Motion values provided below.

Setup & Dependencies

Install the following libraries:

```
npm install motion react react-dom lucide-react
```

```
npm install -D tailwindcss @tailwindcss/vite
```

Ensure Tailwind V4 is correctly initialized via `@tailwindcss/vite` in `vite.config.ts`.

1. Global Styles (`src/index.css`)

Import the necessary Google Fonts and set up the Tailwind V4 `@theme` overrides:

代码块

```
1  @import url('https://fonts.googleapis.com/css2?
    family=Instrument+Serif:ital@0;1&family=Manrope:wght@300;400;500;600&family=Gre
    at+Vibes&display=swap');
2  @import "tailwindcss";
3
4  @theme {
5    --font-serif: "Instrument Serif", serif;
6    --font-sans: "Manrope", sans-serif;
7    --font-script: "Great Vibes", cursive;
8  }
9  2. Orbit Images Component Styles (src/components/OrbitImages.css)
```

This CSS provides the absolute positioning offsets for our custom rotation gallery.

code

CSS

```
.orbit-container {  
  position: relative;  
  margin-left: auto;  
  margin-right: auto;  
}  
  
.orbit-scaling-container {  
  width: 100%;  
  height: 100%;  
  position: relative;  
}  
  
.orbit-scaling-container--responsive {  
  position: absolute;  
  left: 50%;  
  top: 50%;  
  transform-origin: center center;  
}  
  
.orbit-rotation-wrapper {  
  width: 100%;  
  height: 100%;  
  transform-origin: center center;  
  position: relative;  
}  
  
.orbit-path-svg {  
  position: absolute;  
  inset: 0;  
  pointer-events: none;  
}  
  
.orbit-item {  
  position: absolute;  
  will-change: transform;  
  user-select: none;  
}  
  
.orbit-center-content {  
  position: absolute;  
  inset: 0;  
  display: flex;
```

```
56   align-items: center;
57   justify-content: center;
58   z-index: 10;
59 }
```

```
60
61 .orbit-image {
62   width: 100%;
63   height: 100%;
64   object-fit: contain;
65   border-radius: 50%;
66 }
```

67 3. Orbit Images React Component ([src/components/OrbitImages.tsx](#))

68 Create this mathematically precise component that maps motion paths over SVG strings using `offsetPath` and `offsetDistance`. It accepts `Framer Motion` `MotionValues` as overrides to allow the parent `App.tsx` to infinitely control its radius, spread, item size, and rotation during scroll.

```
69
70 code
71 Tsx
72 // @ts-nocheck
73 import { useMemo, useEffect, useRef, useState } from 'react';
74 import { motion, useMotionValue, useTransform, animate, useMotionTemplate }
75   from 'motion/react';
76 import './OrbitImages.css';
77
78 function generateEllipsePath(cx, cy, rx, ry) {
79   return `M ${cx - rx} ${cy} A ${rx} ${ry} 0 1 0 ${cx + rx} ${cy} A ${rx}
80     ${ry} 0 1 0 ${cx - rx} ${cy}`;
81 }
82
83 function generateCirclePath(cx, cy, r) {
84   return generateEllipsePath(cx, cy, r, r);
85 }
86
87 function generateSquarePath(cx, cy, size) {
88   const h = size / 2;
89   return `M ${cx - h} ${cy - h} L ${cx + h} ${cy - h} L ${cx + h} ${cy + h} L
90     ${cx - h} ${cy + h} Z`;
91 }
92
93 function generateRectanglePath(cx, cy, w, h) {
94   const hw = w / 2;
95   const hh = h / 2;
```

```

96 function generateTrianglePath(cx, cy, size) {
97     const height = (size * Math.sqrt(3)) / 2;
98     const hs = size / 2;
99     return `M ${cx} ${cy - height / 1.5} L ${cx + hs} ${cy + height / 3} L ${cx
- hs} ${cy + height / 3} Z`;
100 }
101
102 function generateStarPath(cx, cy, outerR, innerR, points) {
103     const step = Math.PI / points;
104     let path = '';
105     for (let i = 0; i < 2 * points; i++) {
106         const r = i % 2 === 0 ? outerR : innerR;
107         const angle = i * step - Math.PI / 2;
108         const x = cx + r * Math.cos(angle);
109         const y = cy + r * Math.sin(angle);
110         path += i === 0 ? `M ${x} ${y}` : ` L ${x} ${y}`;
111     }
112     return path + ' Z';
113 }
114
115 function generateHeartPath(cx, cy, size) {
116     const s = size / 30;
117     return `M ${cx} ${cy + 12 * s} C ${cx - 20 * s} ${cy - 5 * s}, ${cx - 12 *
s} ${cy - 18 * s}, ${cx} ${cy - 8 * s} C ${cx + 12 * s} ${cy - 18 * s}, ${cx +
20 * s} ${cy - 5 * s}, ${cx} ${cy + 12 * s}`;
118 }
119
120 function generateInfinityPath(cx, cy, w, h) {
121     const hw = w / 2;
122     const hh = h / 2;
123     return `M ${cx} ${cy} C ${cx + hw * 0.5} ${cy - hh}, ${cx + hw} ${cy - hh},
${cx + hw} ${cy} C ${cx + hw} ${cy + hh}, ${cx + hw * 0.5} ${cy + hh}, ${cx}
${cy} C ${cx - hw * 0.5} ${cy + hh}, ${cx - hw} ${cy + hh}, ${cx - hw} ${cy} C
${cx - hw} ${cy - hh}, ${cx - hw * 0.5} ${cy - hh}, ${cx} ${cy}`;
124 }
125
126 function generateWavePath(cx, cy, w, amplitude, waves) {
127     const pts = [];
128     const segs = waves * 20;
129     const hw = w / 2;
130     for (let i = 0; i <= segs; i++) {
131         const x = cx - hw + (w * i) / segs;
132         const y = cy + Math.sin((i / segs) * waves * 2 * Math.PI) * amplitude;
133         pts.push(i === 0 ? `M ${x} ${y}` : `L ${x} ${y}`);
134     }
135     for (let i = segs; i >= 0; i--) {
136         const x = cx - hw + (w * i) / segs;

```

```

137     const y = cy - Math.sin((i / segs) * waves * 2 * Math.PI) * amplitude;
138     pts.push(`L ${x} ${y}`);
139   }
140   return pts.join(' ') + ' Z';
141 }
142
143 function OrbitItem({ item, index, totalItems, pathValue, itemSizeValue,
144   rotationValue, progress, fill, scaleStrength, focalPoint = 50 }) {
145   const itemOffset = fill ? (index / totalItems) * 100 : 0;
146
147   const offsetPercentage = useTransform(progress, (p) => {
148     return (((p + itemOffset) % 100) + 100) % 100;
149   });
150
151   const offsetDistance = useTransform(offsetPercentage, (p) => `${p}%`);
152
153   const itemScale = useTransform(() => {
154     const rawPos = offsetPercentage.get();
155     const strength = scaleStrength ? scaleStrength.get() : 0;
156
157     let dist = Math.abs(rawPos - focalPoint);
158     if (dist > 50) dist = 100 - dist;
159
160     let targetScale = 1;
161     if (dist < 20) {
162       const ratio = dist / 20;
163       const cosCurve = (Math.cos(ratio * Math.PI) + 1) / 2;
164       targetScale = 0.4 + (cosCurve * 0.6);
165     } else {
166       targetScale = 0.4;
167     }
168
169     return 1 - strength * (1 - targetScale);
170   });
171
172   const offsetPath = useMotionTemplate`path("${pathValue}")`;
173
174   return (
175     <motion.div
176       className="orbit-item"
177       style={{
178         width: itemSizeValue,
179         height: itemSizeValue,
180         offsetPath,
181         offsetRotate: '0deg',
182         offsetAnchor: 'center center',
183         offsetDistance,

```

```

183         scale: itemScale,
184         zIndex: useTransform(itemScale, s => Math.round(s * 100)),
185         pointerEvents: 'auto'
186     }}
187     >
188     <motion.div style={{ transform: useTransform(rotationValue, r =>
`rotate(${ -r }deg)`), width: '100%', height: '100%' }}>{item}</motion.div>
189     </motion.div>
190 );
191 }
192
193 export default function OrbitImages({
194     images = [],
195     altPrefix = 'Orbiting image',
196     shape = 'ellipse',
197     customPath,
198     baseWidth = 1400,
199     radiusX = 700,
200     radiusY = 170,
201     radius = 300,
202     starPoints = 5,
203     starInnerRatio = 0.5,
204     rotation = -8,
205     duration = 40,
206     itemSize = 64,
207     direction = 'normal',
208     fill = true,
209     width = 100,
210     height = 100,
211     className = '',
212     showPath = false,
213     pathColor = 'rgba(0,0,0,0.1)',
214     pathWidth = 2,
215     easing = 'linear',
216     paused = false,
217     centerContent,
218     responsive = false,
219     progressOverride,
220     radiusXOverride,
221     radiusYOverride,
222     itemSizeOverride,
223     rotationOverride,
224     translateXOverride,
225     focusStrength,
226 }) {
227     const containerRef = useRef(null);
228     const [scale, setScale] = useState(1);

```

```

229
230     const designCenterX = baseWidth / 2;
231     const designCenterY = baseWidth / 2;
232
233     const currentRadiusX = radiusXOverride || useMotionValue(radiusX);
234     const currentRadiusY = radiusYOverride || useMotionValue(radiusY);
235     const currentItemSize = itemSizeOverride || useMotionValue(itemSize);
236     const currentRotation = rotationOverride || useMotionValue(rotation);
237     const currentTranslateX = translateXOverride || useMotionValue(0);
238
239     const pathValue = useTransform([currentRadiusX, currentRadiusY], ([rx, ry])
=> {
240         switch (shape) {
241             case 'circle': return generateCirclePath(designCenterX, designCenterY,
rx);
242             case 'ellipse': return generateEllipsePath(designCenterX, designCenterY,
rx, ry);
243             case 'square': return generateSquarePath(designCenterX, designCenterY,
rx * 2);
244             case 'rectangle': return generateRectanglePath(designCenterX,
designCenterY, rx * 2, ry * 2);
245             case 'triangle': return generateTrianglePath(designCenterX,
designCenterY, rx * 2);
246             case 'star': return generateStarPath(designCenterX, designCenterY, rx,
rx * starInnerRatio, starPoints);
247             case 'heart': return generateHeartPath(designCenterX, designCenterY, rx
* 2);
248             case 'infinity': return generateInfinityPath(designCenterX,
designCenterY, rx * 2, ry * 2);
249             case 'wave': return generateWavePath(designCenterX, designCenterY, rx *
2, ry, 3);
250             case 'custom': return customPath || generateCirclePath(designCenterX,
designCenterY, rx);
251             default: return generateEllipsePath(designCenterX, designCenterY, rx,
ry);
252         }
253     });
254
255     useEffect(() => {
256         if (!responsive || !containerRef.current) return;
257         const updateScale = () => {
258             if (!containerRef.current) return;
259             setScale(containerRef.current.clientWidth / baseWidth);
260         };
261         updateScale();
262         const observer = new ResizeObserver(updateScale);
263         observer.observe(containerRef.current);

```

```

264     return () => observer.disconnect();
265   }, [responsive, baseWidth]);
266
267   const internalProgress = useMotionValue(0);
268
269   useEffect(() => {
270     if (paused || progressOverride) return;
271     const controls = animate(internalProgress, direction === 'reverse' ? -100
: 100, {
272       duration,
273       ease: easing,
274       repeat: Infinity,
275       repeatType: 'loop',
276     });
277     return () => controls.stop();
278   }, [internalProgress, duration, easing, direction, paused,
progressOverride]);
279
280   const activeProgress = progressOverride || internalProgress;
281   const containerWidth = responsive ? '100%' : (typeof width === 'number' ?
width : '100%');
282   const containerHeight = responsive ? 'auto' : (typeof height === 'number' ?
height : (typeof width === 'number' ? width : 'auto'));
283
284   const items = images.map((src, index) => (
285     <motion.img
286       key={src}
287       src={src}
288       alt={` ${altPrefix} ${index + 1}`}
289       draggable={false}
290       className="orbit-image"
291       whileHover={{ scale: 1.2 }}
292       transition={{ duration: 0.3 }}
293       style={{ cursor: "pointer", pointerEvents: "auto" }}
294     />
295   ));
296
297   return (
298     <div ref={containerRef} className={`orbit-container ${className}`} style=
{{ width: containerWidth, height: containerHeight, aspectRatio: responsive ?
'1 / 1' : undefined }} aria-hidden="true">
299       <div className={responsive ? 'orbit-scaling-container orbit-scaling-
container--responsive' : 'orbit-scaling-container'} style={{ width: responsive
? baseWidth : '100%', height: responsive ? baseWidth : '100%', transform:
responsive ? `translate(-50%, -50%) scale(${scale})` : undefined }}>
300         <motion.div className="orbit-rotation-wrapper" style={{ rotate:
currentRotation, x: currentTranslateX }}>

```

```

301         {showPath && (
302             <svg width="100%" height="100%" viewBox={`0 0 ${baseWidth}
303             ${baseWidth}`} className="orbit-path-svg">
304                 <path d={pathValue.get()} fill="none" stroke={pathColor}
305                 strokeWidth={pathWidth / scale} />
306             </svg>
307         )}
308         {items.map((item, index) => (
309             <OrbitItem key={index} item={item} index={index} totalItems=
310             {items.length} pathValue={pathValue} itemSizeValue={currentItemSize}
311             rotationValue={currentRotation} progress={activeProgress} fill={fill}
312             scaleStrength={focusStrength} focalPoint={50} />
313         ))}
314     </motion.div>
315 </div>
316 {centerContent && <div className="orbit-center-content">{centerContent}
317 </div>}
318 </div>
319 );
320 }

```

321 4. Main Page App Component (src/App.tsx)

322 Implement the exact layout, UI timelines (scrollYProgress transforms),
 323 background <video>, typography mask, and the heavily orchestrated Framer
 324 Motion timeline values. Do not change any numbers in the arrays.

```

325 code
326 Tsx
327 import { motion, useMotionTemplate, useScroll, useTransform,
328 useAnimationFrame, useMotionValue } from 'motion/react';
329 import { useRef } from 'react';
330 import OrbitImages from './components/OrbitImages';
331
332 const orbitImagesData = [
333     "https://res.cloudinary.com/daklr2whx/image/upload/v1776966860/202604232047_gxy
334     qne.jpg",
335     "https://res.cloudinary.com/daklr2whx/image/upload/v1776966856/202604232052_ihy
336     slg.jpg",
337     "https://res.cloudinary.com/daklr2whx/image/upload/v1776966299/15112343_tuzrbg.
338     jpg",
339     "https://res.cloudinary.com/daklr2whx/image/upload/v1776966299/202604232043_vhb
340     6u9.jpg",
341     "https://res.cloudinary.com/daklr2whx/image/upload/v1776967124/02604232058_nh1q

```

```

330     d1.jpg",
331     "https://res.cloudinary.com/daklr2whx/image/upload/v1776967611/202604232105_lv3
332     fhp.jpg",
333 ];
334
335 export default function App() {
336     const containerRef = useRef<HTMLDivElement>(null);
337
338     const { scrollYProgress } = useScroll({
339         target: containerRef,
340         offset: ["start start", "end end"]
341     });
342
343     const rx = useTransform(scrollYProgress, [0, 0.08, 1], ["0%", "55%", "55%"]);
344     const ry = useTransform(scrollYProgress, [0, 0.08, 1], ["0%", "55%", "55%"]);
345     const clipPath = useMotionTemplate`ellipse(${rx} ${ry} at 50% 50%)`;
346
347     const textOpacity = useTransform(scrollYProgress, [0.03, 0.08, 0.15, 0.22,
348     0.90, 0.98, 1], [0, 1, 1, 0, 0, 1, 1]);
349     const textBlurVal = useTransform(scrollYProgress, [0.03, 0.08, 0.15, 0.22,
350     0.90, 0.98, 1], [15, 0, 0, 15, 15, 0, 0]);
351     const filterText = useMotionTemplate`blur(${textBlurVal}px)`;
352     const yElement = useTransform(scrollYProgress, [0.03, 0.08, 0.15, 0.22,
353     0.90, 0.98, 1], [20, 0, 0, 20, 20, 0, 0]);
354
355     const targetRadius = 650;
356
357     const orbitItemSize = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
358     1], [80, 520, 520, 80, 80]);
359     const orbitRx = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
360     1], [330, targetRadius, targetRadius, 330, 330]);
361     const orbitRy = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
362     1], [140, targetRadius, targetRadius, 140, 140]);
363     const orbitRotation = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
364     1], [-15, 0, 0, -15, -15]);
365     const orbitTx = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
366     1], [0, -targetRadius, -targetRadius, 0, 0]);
367     const focusStrength = useTransform(scrollYProgress, [0.15, 0.25, 0.85, 0.95,
368     1], [0, 1, 1, 0, 0]);
369
370     const orbitProgress = useMotionValue(0);
371     const prevScroll = useRef(0);
372
373     useAnimationFrame((time, delta) => {
374         const pos = scrollYProgress.get();
375         const scrollDelta = pos - prevScroll.current;

```

```

365     prevScroll.current = pos;
366
367     let frameSpeed = 0;
368     if (pos > 0.15 && pos < 0.85) {
369         frameSpeed = (scrollDelta * 200);
370     } else {
371         frameSpeed = (delta / 1000) * 2.5;
372     }
373
374     orbitProgress.set(orbitProgress.get() + frameSpeed);
375 });
376
377 return (
378     <div ref={containerRef} className="relative w-full h-[600vh] bg-black">
379         <div className="sticky top-0 w-full h-screen overflow-hidden text-white">
380
381             <video autoPlay loop muted playsInline className="absolute inset-0 w-
full h-full object-cover z-0">
382                 <source
src="https://res.cloudinary.com/daklr2whx/video/upload/v1776960333/a_kitten_dri
nks_202604231827_apoc3w.mp4" type="video/mp4" />
383             </video>
384
385             <div className="absolute inset-0 bg-black/10 z-0"></div>
386
387             <div className="absolute z-10 w-[80vw]" style={{ left: '3vw', bottom:
'3vw' }}>
388                 <svg viewBox="0 0 350 72" className="w-full h-auto drop-shadow-2xl
overflow-visible" preserveAspectRatio="xMinYMax meet">
389                     <text x="-3" y="80" fontFamily="'Instrument Serif', serif"
fill="#FDFFB7" className="select-none">
390                         <tspan fontSize="90">Shamoni</tspan>
391                         <tspan fontSize="28.8" dx="4" dy="-40">@</tspan>
392                     </text>
393                 </svg>
394             </div>
395
396             <motion.div
397                 className="absolute z-20 flex items-center justify-center overflow-
hidden"
398                 style={{ clipPath, rotate: -15, width: '150vw', height: '150vh',
left: '-25vw', top: '-25vh' }}
399             >
400                 <div className="absolute inset-0 bg-white" />
401                 <div className="relative flex flex-col items-center justify-center"
style={{ width: '100vw', height: '100vh', transform: 'rotate(15deg)' }}>

```

```

402         <motion.div className="w-[90vw] max-w-[1200px] aspect-square
relative z-0">
403             <OrbitImages
404                 images={orbitImagesData}
405                 shape="ellipse"
406                 direction="normal"
407                 duration={40}
408                 fill={true}
409                 showPath={false}
410                 responsive={true}
411                 baseWidth={800}
412                 progressOverride={orbitProgress}
413                 radiusXOverride={orbitRx}
414                 radiusYOverride={orbitRy}
415                 itemSizeOverride={orbitItemSize}
416                 rotationOverride={orbitRotation}
417                 translateXOverride={orbitTx}
418                 focusStrength={focusStrength}
419             />
420         </motion.div>
421     </div>
422 </motion.div>
423
424     <div className="absolute inset-0 z-[60] pointer-events-none">
425         <div className="absolute top-[48%] left-1/2 -translate-x-1/2 -
translate-y-1/2 pointer-events-none z-50">
426             <motion.div
427                 className="flex flex-col items-center whitespace-nowrap
pointer-events-auto"
428                 style={{ filter: filterText, opacity: textOpacity,
WebkitFontSmoothing: 'antialiased', WebkitBackfaceVisibility: 'hidden',
transform: 'translateZ(0)' }}
429             >
430                 <div className="flex items-baseline text-black leading-none mb-
1">
431                     <span className="font-serif text-[45px] md:text-[55px]
italic tracking-tight text-black">M</span>
432                     <span className="font-serif text-[45px] md:text-[55px]
tracking-tight text-black">aster the Elements</span>
433                 </div>
434                 <span className="font-sans text-[28px] md:text-[36px] tracking-
tight text-black mt-[-5px]">embrace</span>
435             </motion.div>
436         </div>
437
438     </motion.div>

```

```

439         className="absolute top-32 right-[calc(6vw+150px)] md:right-
[214px] flex flex-col items-start text-left pointer-events-auto cursor-text"
440         style={{ y: yElement, filter: filterText, opacity: textOpacity }}
441     >
442     <span className="font-serif text-[40px] leading-none mb-3 text-
black">2K26</span>
443     <span className="font-serif text-[16px] uppercase tracking-
widest text-black leading-[20px] text-left">
444         JOIN AN EXCLUSIVE<br />COMMUNITY
445     </span>
446 </motion.div>
447
448 <motion.div
449     className="absolute bottom-8 left-8 md:bottom-16 md:left-16 flex
flex-col items-start text-black pointer-events-auto cursor-text"
450     style={{ y: yElement, filter: filterText, opacity: textOpacity }}
451 >
452     <span className="font-serif text-[40px] leading-none mb-1 text-
black">0651</span>
453     <span className="font-serif text-[16px] uppercase tracking-
widest text-black">COLLECTION</span>
454 </motion.div>
455
456 <div className="absolute bottom-16 right-[6vw] md:right-[10vw]
flex flex-col items-start z-10 pointer-events-auto">
457     <motion.p
458         className="font-serif text-[16px] uppercase tracking-widest
text-black leading-[20px] mb-6 text-left w-[240px] cursor-text"
459         style={{ y: yElement, filter: filterText, opacity: textOpacity
}}
460     >
461     JOIN AN EXCLUSIVE COMMUNITY OF SAILORS. WHETHER YOU CRAVE THE
THRILL OF THE OPEN
462     </motion.p>
463     <motion.div className="flex gap-0 pointer-events-auto items-
center" style={{ y: yElement, filter: filterText, opacity: textOpacity }}>
464         <button className="bg-black hover:bg-black/90 transition-
colors text-white rounded-[40px] px-8 py-3.5 font-serif tracking-[0.1em]
uppercase text-[12px] md:text-[14px] z-10">
465             BUY COLLECTION
466         </button>
467         <button className="bg-black hover:bg-black/90 transition-
colors w-[46px] h-[46px] flex items-center justify-center rounded-[50%] text-
white -ml-2 z-0">
468             <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="ml-1">

```

```

469         <path d="M5 12h14M12 5l7 7-7 7"/>
470     </svg>
471 </button>
472 </motion.div>
473 </div>
474 </div>
475
476 <motion.header
477     className="fixed top-0 left-0 w-full p-6 md:p-10 flex justify-
between items-start z-[100] pointer-events-none"
478     style={{ opacity: textOpacity, filter: filterText }}
479 >
480     <div className="flex items-start text-black select-none leading-none
pointer-events-auto" style={{ fontFamily: "'Instrument Serif', serif",
WebkitFontSmoothing: "antialiased" }}>
481         <span style={{ fontSize: '40px' }}>Shamoni</span>
482         <span style={{ fontSize: '14px', marginLeft: '4px', marginTop:
'4px' }}>@</span>
483     </div>
484
485     <button className="group relative flex items-center justify-center w-
[72px] h-[44px] hover:scale-105 transition-transform duration-300 cursor-
pointer pointer-events-auto" aria-label="Menu">
486         <div className="absolute inset-0 bg-black rounded-[50%] -rotate-
15"></div>
487         <svg className="relative z-10" width="24" height="10" viewBox="0 0
24 10" fill="none" xmlns="http://www.w3.org/2000/svg">
488             <path d="M1 1H23M1 9H23" stroke="white" strokeWidth="2"
strokeLinecap="round" />
489         </svg>
490     </button>
491 </motion.header>
492
493 </div>
494 </div>
495 );
496 }

```